

实验 8. Python 数据分析

网安 2301 班 学号 23110111 姓名 吕仔贤

1. 实验目的：

综合运用所学的 Python 基础知识，结合相关领域的第三方模块，进行综合程序设计，解决实际问题。

2. 实验平台：

Windows 系列+Python3.X

3. 主要实验内容：

一、完成下列验证性程序内容

1. 解线性方程组

使用 SciPy 求解下面的线性方程组：

$$\begin{cases} 2w + x - 5y + z = 8 \\ w - 3x - 6z = 9 \\ 2x - y + 2z = -5 \\ w + 4x - 7y + 6z = 0 \end{cases}$$

```
import numpy as np
import scipy
from scipy import linalg

a= np.mat(' [2 1 -5 1;1 -3 0 -6;0 2 -1 2;1 4 -7 6]')
b=np.mat(' [8;9;-5;0]')
solve = linalg.solve(a, b)

print(solve)
```

运行结果为：

```
[[ 3.]
 [-4.]
 [-1.]
 [ 1.]]
>>>
```

即方程组的解为

$$\begin{cases} w = 3 \\ x = -4 \\ y = -1 \\ z = 1 \end{cases}$$

2.图像处理

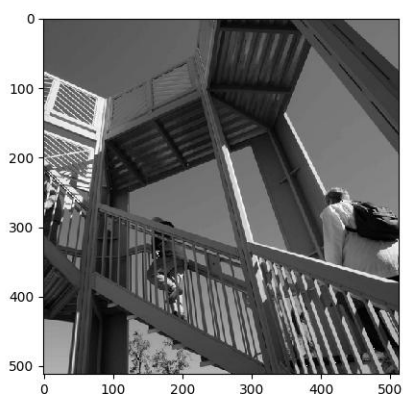
使用 SciPy.ndimage 对图像进行处理。

```
from scipy import ndimage
from scipy import misc
import pylab as pl
ascent = misc.ascent()

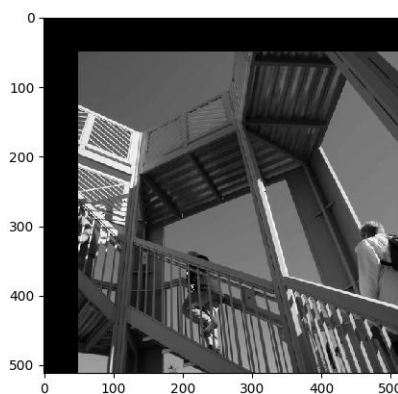
shifted_ascent = ndimage.shift(ascent, (50, 50))
shifted_ascent2 = ndimage.shift(ascent, (50, 50), mode="nearest")
rotated_ascent = ndimage.rotate(ascent, 30)

pl.imshow(ascent, cmap=pl.cm.gray)
pl.figure()
pl.imshow(shifted_ascent, cmap=pl.cm.gray)
pl.figure()
pl.imshow(shifted_ascent2, cmap=pl.cm.gray)
pl.figure()
pl.imshow(rotated_ascent, cmap=pl.cm.gray)
pl.show()
```

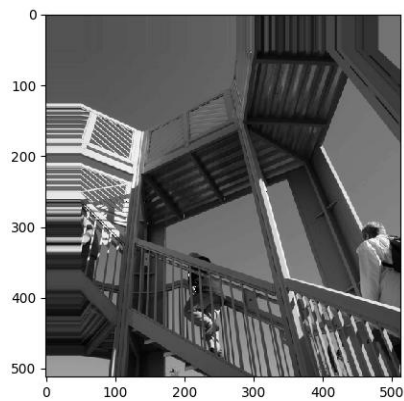
其中，`ascent = misc.ascent()` 生成了一个 SciPy 库自带的灰度图片；语句 `ndimage.shift(ascent, (50, 50))` 对图片进行了平移处理，处理后的图片如图 9-1 (b) 所示；语句 `ndimage.shift(ascent, (50, 50), mode="nearest")` 为平移后自动填充，处理效果如图 9-1 (c) 所示；语句 `rotated_ascent = ndimage.rotate(ascent, 30)` 对图片做了逆时针旋转 30° 的处理，处理后的图片如图 9-1 (d) 所示。



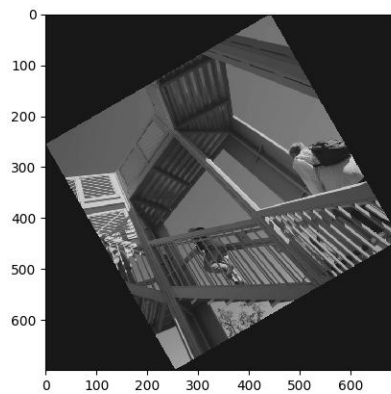
(a) 预处理灰度图片



(b) 平移处理后的图片（未自动填充）



(c) 平移处理后的图片（自动填充）



(d) 旋转处理后的图片

图 9-1 使用 SciPy 库对图片进行处理

3. 使用 matplotlib 画图。

```
import matplotlib.pyplot as plt
import numpy as np
```

```
x = np.linspace(0, 10, 1000) # 设置自变量格式
y = np.sin(x) + 1           # 设置因变量 y
z = np.cos(x**2) + 1        # 设置因变量 z
```

```
plt.figure(figsize=(8, 4)) # 设置图像大小
plt.plot(x, y, label="sinx+1", color='red', linewidth=2) # 作图 (x, y), 设置标签格式
plt.plot(x, z, label="cosx^2+1") # 作图 (x, z)
plt.xlabel('Time(s)') # 设置 x 轴名称
plt.ylabel('Volt') # 设置 y 轴名称
plt.title('A simple Example') # 设置表格标题
plt.ylim(0, 2.2) # 显示的 y 轴范围
plt.legend() # 显示图例
plt.show() # 显示作图结果
```

运行上面的代码，会弹出一个如图 9-2 所示的窗口。单击保存图标可以将表格保存为 .eps、.jpg、.pdf、.png、.raw 等图片格式。保存的图片质量几乎可以满足各种出版要求。

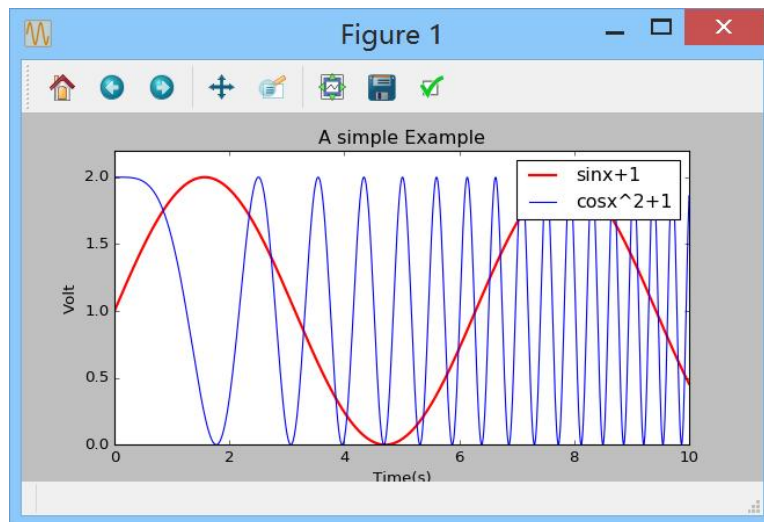


图 9-2 使用 Matplotlib 画图

注意：

- 实际绘图时可能会发现中文无法正常显示的情况，这是因为 Matplotlib 的默认字体为英文字体所致，需要在作图之前指定默认字体为中文。代码如下：

```
plt.rcParams['font.sans-serif'] = ['SimHei']
```

- 如果在保存作图图像时发现负号显示不正常，可以通过下面的代码解决：

```
plt.rcParams['axes.unicode_minus'] = False
```

4. 使用 matplotlib 实现数据可视化。

```
D:\教学2020下半年\Python\python-3.8.5>pip install xlrd
Collecting xlrd
  Downloading xlrd-1.2.0-py2.py3-none-any.whl (103 kB)
  | 103 kB 142 kB/s
Installing collected packages: xlrd
Successfully installed xlrd-1.2.0
```

```
import matplotlib.pyplot as plt
import xlrd
```

```
data= xlrd.open_workbook("data.xls")
sh = data.sheet_by_name("Sheet1")
x=sh.col_values(0)
y=sh.col_values(1)
plt.plot(x, y, '.')
plt.show()
```

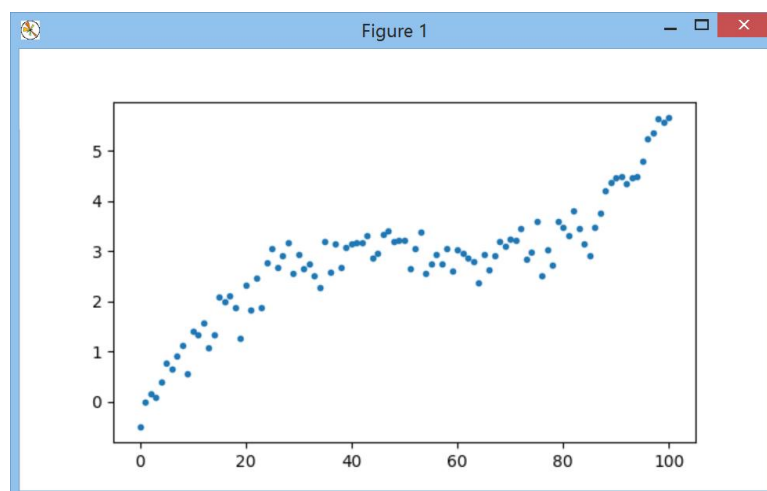


图 9-3 使用 Python 进行数据可视化

上面的代码是一个从 Excel 文件中读取数据，并使用 Python 实现数据可视化的例子，其运行结果如图 9-3 所示。

通过上面的几个例子我们可以看到，使用 Python 语言，仅仅通过短短的几行代码（少则五六行，多则十几行），即可实现方程组求解、图像处理、绘图、数据可视化等非常复杂的操作。这是其他编程语言望尘莫及的。

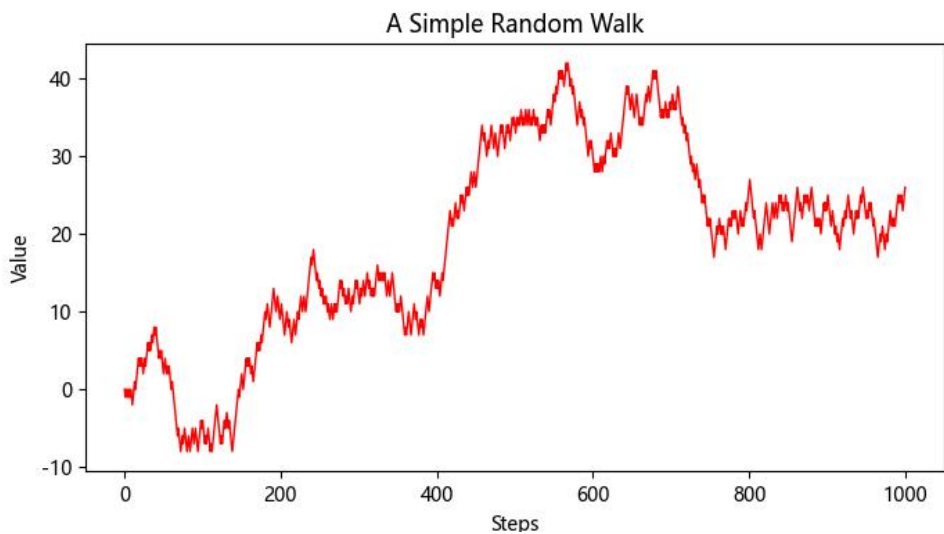
5. 从 0 开始，步长值为 1 或者 -1，且出现的概率相等，通过使用 Python 内置的 random 实现 1000 步的随机漫步，并使用 Matplotlib 生成折线图。

```
import random
import matplotlib.pyplot as plt
import numpy as np

position=0
walk = [position]
steps = 1000
for i in range(steps):
    step = 1 if random.randint(0,1) else -1
    position +=step
    walk.append(position)

plt.figure(figsize=(8, 4)) # 设置图像大小
plt.plot(walk, color='red', linewidth=1) # 作图 (x,y)，设置标签格式
plt.xlabel('Steps') # 设置 x 轴名称
plt.ylabel('Value') # 设置 y 轴名称
plt.title('A Simple Random Walk') # 设置表格标题
plt.show() # 显示作图结果
```

运行结果：



二、编程实现销售数据的统计分析及可视化

给定销售数据

```
np.random.seed(42)
sales_data = {
    '日期': pd.date_range('2024-01-01', periods=100, freq='D'),
    '产品': np.random.choice(['A', 'B', 'C', 'D'], 100),
    '销售额': np.random.randint(100, 5000, 100),
    '数量': np.random.randint(1, 20, 100),
    '地区': np.random.choice(['北京', '上海', '广州', '深圳'], 100)
}

sales_df = pd.DataFrame(sales_data)
```

要求：

1. 筛选出销售额大于 2000 的记录
2. 计算每个产品的总销售额和平均销售额
3. 找出每个地区销售额最高的产品
4. 计算每周的销售总额

使用题目的 sales_df 数据

要求：

1. 绘制各产品销售额的柱状图
2. 绘制销售额随时间变化的折线图
3. 绘制各地区销售额占比的饼图
4. 绘制销售额与数量的散点图

源代码:

```
import pandas as pd
import numpy as np
import matplotlib.pyplot as plt

# ===== 1. 基础设置 =====
# 设置绘图风格 (ggplot 风格, 简洁美观)
plt.style.use('ggplot')
# 解决中文显示和负号问题
plt.rcParams['font.sans-serif'] = ['SimHei']
plt.rcParams['axes.unicode_minus'] = False
# 设置 Pandas 控制台打印格式 (让输出对齐, 看起来舒服)
pd.set_option('display.unicode.ambiguous_as_wide', True)
pd.set_option('display.unicode.east_asian_width', True)
pd.set_option('display.width', 180)
pd.set_option('display.max_columns', None)
pd.options.display.float_format = '{:.2f}'.format # 小数保留 2 位
# ===== 2. 数据生成 =====
np.random.seed(42)
sales_data = {
    '日期': pd.date_range('2024-01-01', periods=100, freq='D'),
    '产品': np.random.choice(['A', 'B', 'C', 'D'], 100),
    '销售额': np.random.randint(100, 5000, 100),
    '数量': np.random.randint(1, 20, 100),
    '地区': np.random.choice(['北京', '上海', '广州', '深圳'], 100)
}
sales_df = pd.DataFrame(sales_data)
# ===== 3. 数据处理与输出 =====
# --- 要求 1: 筛选出销售额大于 2000 的记录 (导出为 Excel) ---
high_sales = sales_df[sales_df['销售额'] > 2000]
# index=False 表示不把左边那列序号存进去, 表格更干净
high_sales.to_excel('销售额大于 2000 的记录.xlsx', index=False)
print(f">>> 已将 {len(high_sales)} 条高销售额记录导出为 Excel 文件。")
# --- 要求 2: 计算每个产品的总销售额和平均销售额 ---
print("\n>>> 各产品销售统计:")
product_stats = sales_df.groupby('产品')['销售额'].agg(['sum', 'mean'])
print(product_stats)
# --- 要求 3: 找出每个地区销售额最高的产品 ---
print("\n>>> 各地区销售额最高的产品:")
region_group = sales_df.groupby(['地区', '产品'])['销售额'].sum().reset_index()
best_products = region_group.sort_values('销售额', ascending=False).drop_duplicates(['地区'])
print(best_products.sort_values('地区'))
# --- 要求 4: 计算每周的销售总额 ---
```

```

print("\n>>> 每周销售总额 (前5周示例):")
df_temp = sales_df.set_index('日期')
weekly_sales = df_temp.resample('W')['销售额'].sum()
print(weekly_sales.head())

# ===== 4. 绘图与保存 =====

# 定义一套清新的配色方案
colors = ['#5DADE2', '#AF7AC5', '#48C9B0', '#F4D03F', '#EB984E']

# 图1:各产品销售额柱状图
plt.figure(figsize=(10, 6))
ax1 = product_stats['sum'].plot(kind='bar', color=colors[:4], width=0.6, edgecolor='white')
plt.title('各产品销售额统计', fontsize=14)
plt.ylabel('总销售额', fontsize=12)
plt.xlabel('产品型号', fontsize=12)
plt.xticks(rotation=0)

# 柱子上标数值
for p in ax1.patches:
    ax1.annotate(str(int(p.get_height()))), (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', xytext=(0, 5), textcoords='offset points')
plt.tight_layout()
plt.savefig('1_产品销售额_美化版.png', dpi=300)
plt.close()

# 图2:销售额随时间变化折线图
plt.figure(figsize=(12, 6))
daily_sales = sales_df.groupby('日期')['销售额'].sum()
plt.plot(daily_sales.index, daily_sales.values, color='#E74C3C', linewidth=2, marker='o', markersize=4)
plt.title('2024 年销售额趋势', fontsize=14)
plt.xlabel('日期')
plt.ylabel('日销售额')
plt.grid(True, linestyle='--', alpha=0.7)
plt.tight_layout()
plt.savefig('2_时间趋势_美化版.png', dpi=300)
plt.close()

# 图3:各地区占比饼图
plt.figure(figsize=(8, 8))
region_sales = sales_df.groupby('地区')['销售额'].sum()
explode = [0.03] * len(region_sales) # 让饼图裂开一点点缝隙
plt.pie(region_sales, labels=region_sales.index, autopct='%1.1f%%',
        startangle=140, colors=colors, explode=explode, textprops={'fontsize': 12})
plt.title('各地区销售额占比', fontsize=14)
plt.tight_layout()
plt.savefig('3_地区占比_美化版.png', dpi=300)
plt.close()

# 图4:销售额与数量散点图
plt.figure(figsize=(10, 6))

```



```
plt.scatter(sales_df['数量'], sales_df['销售额'], color='#8E44AD', alpha=0.6, s=60, edgecolors='white')
plt.title('销售额 vs 销售数量', fontsize=14)
plt.xlabel('数量', fontsize=12)
plt.ylabel('销售额', fontsize=12)
plt.tight_layout()
plt.savefig('4_散点分析_美化版.png', dpi=300)
plt.close()
print("\n>>> 所有图表绘制完成, 已保存至文件夹。")
```

运行结果:

● >>> 已将 62 条高销售额记录导出为 Excel 文件。

>>> 各产品销售统计:

	sum	mean
产品		
A	63652	3182.60
B	65480	2518.46
C	55168	2298.67
D	81857	2728.57

>>> 各地区销售额最高的产品:

	地区	产品	销售额
2	上海	C	17114
5	北京	B	23959
11	广州	D	28851
15	深圳	D	26414

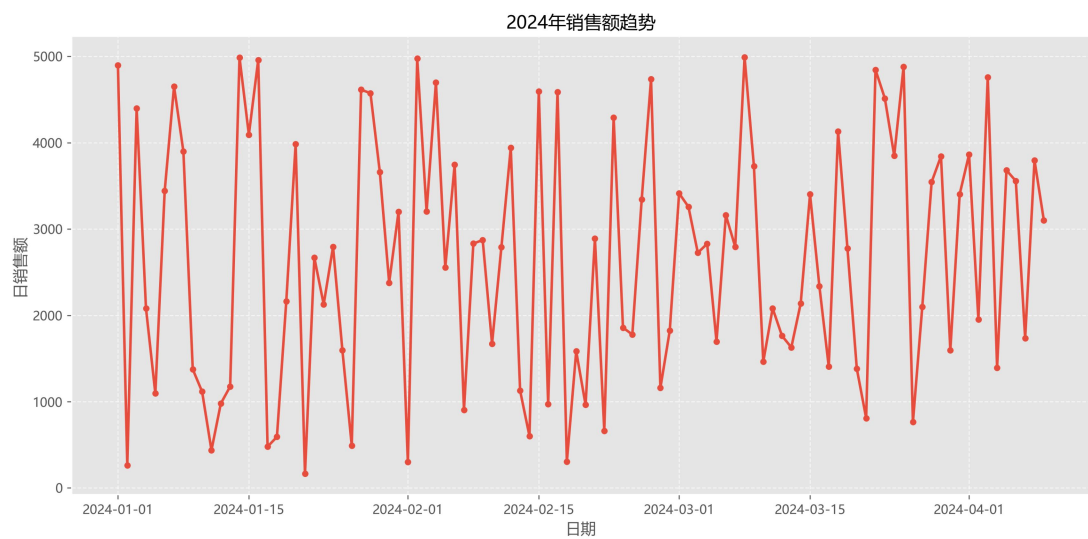
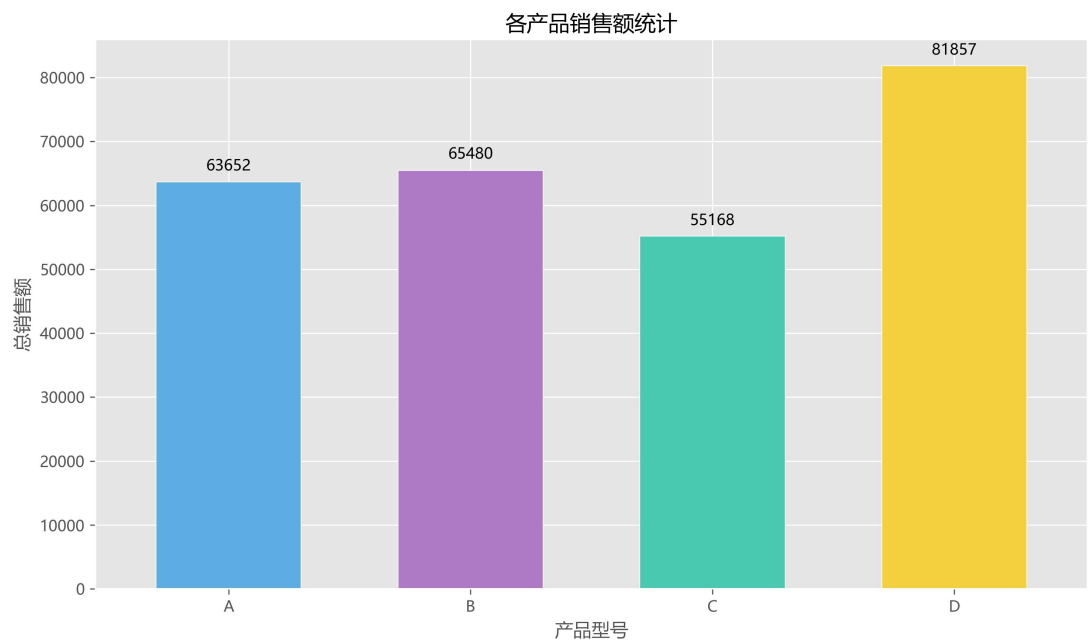
>>> 每周销售总额 (前5周示例):

日期	
2024-01-07	20825
2024-01-14	13967
2024-01-21	16433
2024-01-28	18864
2024-02-04	22415

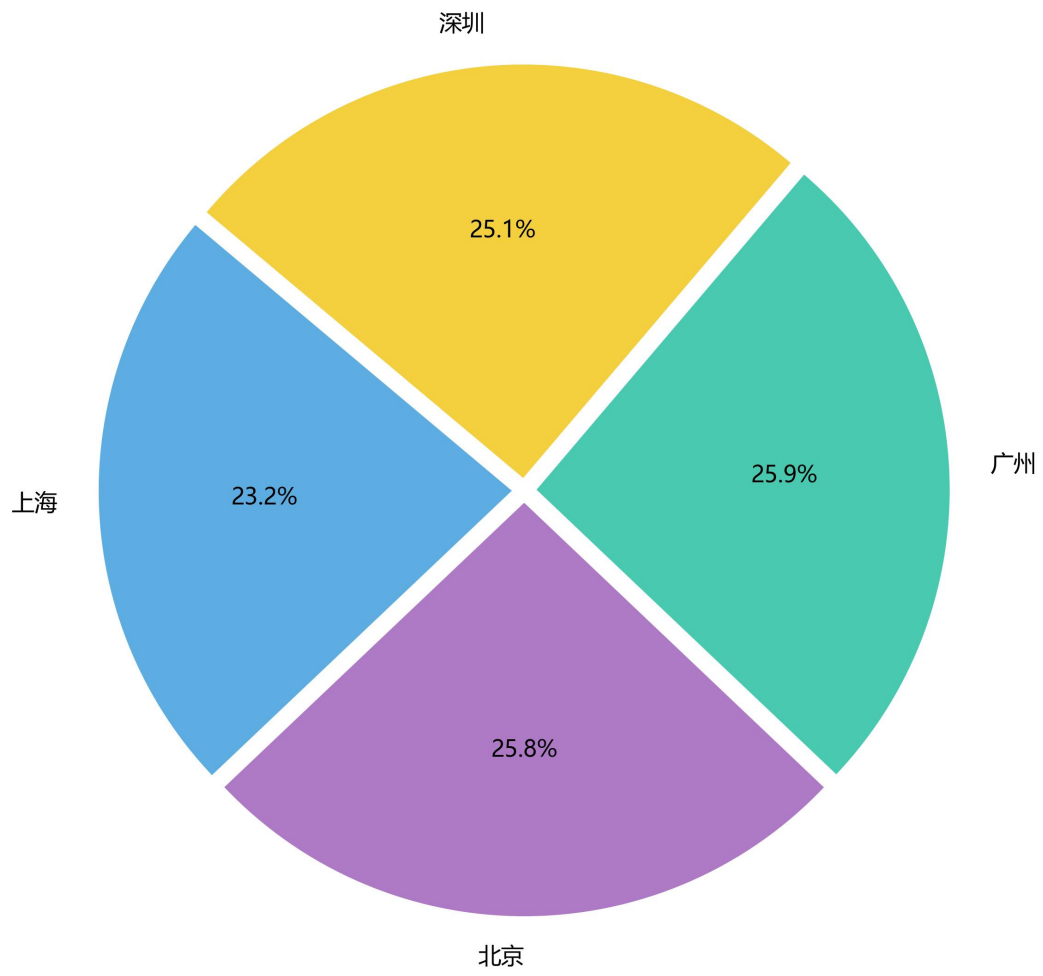
Freq: W-SUN, Name: 销售额, dtype: int32

>>> 所有图表绘制完成, 已保存至文件夹。

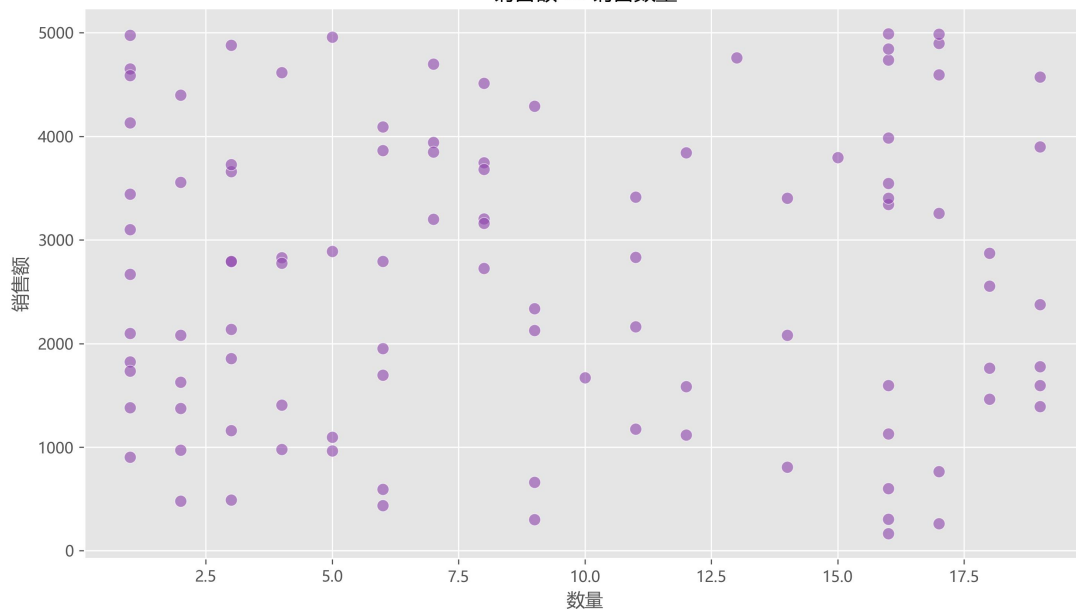
(销售额大于 2000 的记录具体见”附件——销售额大于 2000 的记录.xlsx”)



各地区销售额占比



销售额 vs 销售数量



4. 实验总结

通过本次实验，我系统地掌握了 Python 数据分析的核心流程，主要收获如下：

➤ 工具库的协同运用

深入体会了 NumPy、Pandas 和 Matplotlib 这“数据分析三剑客”的配合。特别是 Pandas 的 `groupby`（分组）和 `resample`（重采样）功能，在处理销售数据的多维统计时，比传统 Excel 操作效率高很多，且代码具有很好的复用性。

➤ 数据可视化的细节掌控

在绘图过程中，我实际遇到了中文显示为方块、负号显示异常的问题，通过配置 `rcParams` 成功解决。我也学会了使用 `ggplot` 风格来美化图表，意识到数据分析不仅要结果准确，图表的清晰美观同样重要。

➤ 从数据到洞察

通过将枯燥的销售数字转化为柱状图、折线图和饼图，我直观地看到了产品销量的趋势和地区占比。这次实验让我明白了 Python 不仅仅是写代码的工具，更是挖掘数据背后规律的利器。